



软件开发类课程考查报告

学院名称: 计算机与信息工程学院

课程名称: 移动应用开发

班级名称: 软件 232

学生姓名: 王义琳

学 号: 2023210632

题 目: 适老居家生活辅助系统

指导教师: 刘东良

起止日期: 2026 年 6 月 1 日至 2026 年 6 月 21 日

计算机软件课程组 制

指导教师评语

该生提交的《线上购物平台开发与整合》报告，选题紧密贴合移动应用开发课程“理论结合实践、聚焦跨平台技术应用”的目标，聚焦线上购物场景的全流程技术实现，具有较强的实际应用价值。报告结构完整且逻辑清晰，涵盖项目背景、需求分析、技术选型及分阶段开发过程，各模块衔接自然。技术应用上，该生熟练运用 Cordova 相关技术栈，完成了各模块的衔接，实现了与原生插件的通信适配，望后续加强 Cordova 与前端框架的深度整合研究，优化页面渲染效率，结合混合开发场景进一步提升用户交互体验，增强技术方案的创新性与实用性。

成绩评定

序号	类型	成绩	权重	得分
1	项目		30%	
2	小组		20%	
3	个人		20%	
4	答辩		30%	
5	总成绩		100%	

指导教师签名：

填表日期： 2026 年 6 月 21 日

目录

指导教师评语.....	2
成绩评定.....	2
1 项目概述	1
1.1 项目背景与来源	1
1.2 项目技术栈	1
1.3 系统架构设计	2
1.4 个人角色与职责.....	3
2 技术架构设计.....	4
2.1 整体架构.....	4
2.2 系统架构图.....	5
2.3 Cordova 客户端技术栈.....	7
2.3.1 技术选型对比报告.....	7
2.3.2 开发环境.....	8
2.4 后端服务技术栈.....	9
2.5 数据库设计.....	10
2.6 API 设计.....	12
2.7 系统核心类图.....	12
2.8 核心功能顺序图.....	15
2.8.1 AI 智能对话（多模型降级）.....	15
2.8.2 用药提醒（本地存储 + 云端同步）.....	17
2.8.3 健康监测（并行查询 + 趋势图）.....	19
2.8.4 紧急呼叫（SOS 长按倒计时）.....	21
3 个人负责功能实现.....	23

3.1 插件化模块系统.....	23
3.2 AI 智能聊天助手（多模型降级）.....	24
3.3 健康监测模块.....	26
3.4 用药提醒模块.....	28
3.5 紧急呼叫（SOS）模块.....	30
3.6 通知推送系统.....	32
3.7 语音交互系统.....	33
3.8 消息中心与智能家居模块.....	34
4 团队协作与跨平台整合.....	35
4.1 协作模式.....	35
4.2 四阶段开发过程.....	36
4.3 跨平台功能对比.....	36
4.4 Cordova 端与其他 5 端联调细节.....	37
4.5 跨平台数据同步方案.....	37
4.6 各成员贡献统计.....	38
5 性能优化与质量保证.....	39
5.1 UI 渲染优化.....	39
5.2 网络层优化.....	39
5.3 数据库查询优化.....	39
5.4 后端性能优化.....	39
5.5 Bug 修复记录.....	39
5.6 测试覆盖.....	40
6 部署与交付.....	41
6.1 后端部署.....	41
6.2 Cordova Android APK 构建.....	41

6.3 交付物清单.....	42
7 创新点与技术亮点.....	42
7.1 多 AI 模型降级链与本地意图优先.....	42
7.2 纯手工讯飞 IAT 协议与原生能力扩展.....	43
7.3 API-First 跨平台数据同步与离线优先策略.....	43
7.4 答辩核心问题解答与演示记录.....	43
7.4.1 答辩核心问题.....	43
7.4.2 答辩现场记录.....	44
7.5 AI 工具创新应用.....	45
8 项目总结与展望.....	45
8.1 项目成果总结.....	45
8.2 技术能力提升.....	46
8.3 经验教训.....	46
8.3.1 已知功能限制.....	47
8.4 未来扩展方向与改进设计.....	47
8.5 进一步发展方向.....	48
8.6 个人反思与后续规划.....	48

1 项目概述

1.1 项目背景与来源

在人口老龄化加剧与移动互联网快速普及的双重驱动下，老年群体对智能化居家生活辅助的需求日益迫切。然而，现有移动应用普遍存在字体偏小、操作复杂、功能冗余等问题，难以有效覆盖独居老人在健康监测、用药提醒、紧急求助等高频场景下的核心需求。针对上述适老化使用痛点，本项目面向居家养老场景，设计并实现一款覆盖 6 种主流移动技术栈、具备跨端协同能力的“EHLAS 适老居家生活辅助系统”。

本项目为移动应用开发课程的综合实践项目，由 6 名成员分别采用 6 种不同技术栈（Android 原生、微信小程序、鸿蒙 ArkTS、.NET MAUI、Uniapp、Cordova）并行开发同一业务场景，并基于统一后端 API 实现跨平台数据互通。

在技术实践层面，通过多端并行开发同一业务场景，团队成员得以深入掌握各类移动开发框架与跨平台技术。在团队协作层面，项目依托统一后端 API、标准化数据模型与每日站会机制，有效锻炼了成员在需求分析、系统设计、Git 协作及项目管理等方面的软件工程实践能力。在社会应用层面，项目构建集健康监测、AI 智能对话、用药提醒、紧急呼叫于一体的综合辅助系统，为适老化移动应用的设计与落地提供了可资借鉴的实践样本。

1.2 项目技术栈

本 EHLAS 适老居家生活辅助系统的开发以打造功能完善、性能稳定、跨端一致的适老化移动应用为总体目标，聚焦六大具体目标：

1. 功能完整性：覆盖用户注册登录、健康监测、AI 智能对话、用药提醒、紧急呼叫、位置追踪等老年用户高频使用的核心场景，整体功能完整度达到 98%。

2. 多端覆盖：通过 Android 原生、微信小程序、鸿蒙 ArkTS、.NET MAUI、Uniapp、Cordova 6 个主流移动技术栈并行实现客户端，统一对接 `http://47.96.4.21:3001` 的 Node.js + Express 后端 RESTful API。

3. 适老化设计：每个平台落实大字体（最小 18sp）、高对比度、简化导航、TTS 语音播报与语音输入等无障碍能力。

4. 性能优化：冷启动时间 ≤ 3 秒（实测约 2.5s）、API 响应 $\leq 500\text{ms}$ （实测约 280ms）、内存占用 $\leq 200\text{MB}$ （实测 95 - 180MB）。

5. AI 与跨端同步：集成 DeepSeek、GLM、讯飞、MiMo 等多家 AI 服务并实现自动降级，跨端同步成功率达到 99.5%。

6. 安全性与可扩展性：通过 AI 模型密钥 XOR 加密、HTTPS 传输、JWT 鉴权保障用户隐私，并借助插件化模块架构为后续扩展提供便利。

成功标准：项目六大目标的达成情况如表 1-1 所示。

表 1-1 项目成功标准达成情况表

标准	目标值	实际达成	状态
功能完整性	100%	98%	达成
平台覆盖	6 个平台	6 个	达成
冷启动时间	<3s	约 2.5s	达成
内存占用	<200MB	约 95-180MB	达成
测试覆盖率	80%	65%	接近达成（注）
Bug 修复率	95%	93%	接近达成（注）

注：测试覆盖率偏低主要因为 Cordova 插件（SQLite、语音识别、TTS、通知等）强依赖 Android 真机环境，无法在 Node.js 测试框架中直接运行。纯逻辑模块（PluginManager、common.js、AIService、数据校验函数）已通过 Jest 覆盖 22 个自动化用例，通过率 100%。Bug 修复率 93%（57 个已修复 53 个），剩余 4 个 Bug 为 Cordova 插件兼容性遗留问题，已纳入后续迭代计划。

1.3 系统架构设计

团队由 6 名成员组成，分别负责 6 个不同的移动技术栈，具体分工如表 1-2 所示。

表 1-2 团队组成与分工表

序号	姓名	学号	主要技术栈	主要职责	联系方式
1	陈琪琪	2023210580	微信小程序 + Node.js 后端	项目经理、健康数据展示编辑、图表绘制、AI 健康提醒、用药提醒、后端 API 开发	19565991631
2	钱家澍	2023210618	Android 原生 (Java)	位置追踪、AI 异常检测、语音助手、紧急求助、数据同步	15255153587
3	何妍婷	2023210593	鸿蒙 ArkTS + Node.js 后端	用户认证、健康监测、设备管理、主题切换、后端 API 开发	18815753969
4	刘海静	2023210607	.NET MAUI (C#)	健康仪表盘、健康图表、AI 趋势分	15155122343

5	车蕊	2023210578	Uniapp (Vue.js)	析、健康报告生成 AI 语音助手、地 图导航、紧急呼 叫、高级组件应用	19565992501
6	王义琳	2023210632	Cordova (HTML/CSS/ JS)	AI 聊天助手、用 药提醒、紧急呼 叫、健康监测、插 件化架构	17334598930

项目管理角色：

项目经理：陈琪琪

技术负责人：何妍婷（后端）、各平台成员独立负责本端技术

质量负责人：全体成员共同负责

文档负责人：全体成员各自负责本平台文档

1.4 个人角色与职责

个人角色定位： 前端开发工程师 + Cordova 混合应用开发与模块集成

个人负责技术栈： Apache Cordova (HTML / CSS / JavaScript)

Cordova 允许使用 HTML、CSS、JavaScript 构建 Android 应用，并通过插件桥接本地数据库、语音识别、TTS、通知、权限等原生能力，适合功能模块多、交互需要快速迭代的混合应用场景。个人技术栈在项目中的具体应用如表 1-3 所示。

表 1-3 个人技术栈应用表

技术类别	具体技术	在项目中的应用
开发语言	JavaScript / HTML5 / CSS3	页面交互、业务逻辑、适老化界面实现
UI 框架	原生 Web 页面 + Cordova WebView	多页面 UI、聊天浮层、业务页面展示
状态管理	页面状态 + localStorage / sessionStorage	登录态、未读状态、页面临时状态保存
网络库	fetch + cordova-plugin-advanced-http	调用后端接口、同步健康/用药/联系人数据
本地存储	cordova-sqlite-storage + localStorage 降级	健康记录、用药计划、联系人、消息、本地缓存
第三方库	TTS、语音识别、本地通知、权限插	语音输入、语音播报、用药提

个人负责的系统功能模块：

1. Cordova 端插件化模块系统（核心功能）
2. AI 聊天助手（多模型、语音输入、语音播报、安全拦截）（核心功能）
3. 健康监测、用药提醒、紧急呼叫（本地存储、后端同步、通知提醒）（核心功能）
4. 消息中心、智能家居、适老化 UI 组件库

2 技术架构设计

2.1 整体架构

架构模式：**分层架构 + 统一后端服务架构。**

本项目采用“多端并行 + 统一后端”的架构模式，6 种技术栈客户端通过 RESTful API / HTTP 方式统一接入部署在阿里云 47.96.4.21:3001 的 Node.js + Express 后端服务，由后端集中管理业务逻辑、数据存储与第三方服务调用，保证多端业务一致性与数据互通。

架构层次说明：

移动端层（Multi-Platform Clients）：包含 6 种技术栈的客户端应用，覆盖 Android、Windows、鸿蒙、Web 等多端环境。

1. Android 原生（Java）：MVVM + 分层架构，承担位置追踪、AI 异常检测、语音助手与紧急求助
2. Uniapp（Vue）：多端适配架构，基于 uni-ui 组件库实现地图导航、AI 语音交互
3. 微信小程序（WXML）：原生小程序架构，承担健康数据管理、图表、AI 咨询与用药提醒
4. 鸿蒙 ArkTS：声明式 ArkUI + 分布式数据管理架构，支持健康检测、设备联动、多端数据共享
5. .NET MAUI（C#）：MVVM + 分层架构，覆盖 Android 与 Windows 双端，承担数据报表与 AI 趋势分析
6. Cordova（HTML/JS）：分层架构 + 自研插件化模块架构，承担适老化组件、AI 聊天助手、语音链路 with 插件按需动态加载

后端服务层：Node.js + Express + SQL.js，提供 43 个 RESTful API 接口，JWT 鉴权，部署于阿里云。

数据层：SQLite 服务端数据库 + 各端本地存储。

第三方服务层：AI 服务（DeepSeek / GLM / 讯飞星火 / MiMo）、百度地图、百度语音。

2.2 系统架构图

整体系统架构如图 2-1 所示。



图 2-1 EHLAS 系统整体架构图

如图 2-1 所示，项目整体采用四层架构：最上层为移动端层，包含 Android、微信小程序、鸿蒙、MAUI、Uniapp、Cordova 六个客户端，各自独立开发但通过统一的 RESTful API 与后端通信；中间为后端服务层，基于 Node.js + Express 提供 43 个 API 接口，通过 JWT 进行统一身份鉴权；下方为数据层，服务端使用 SQLite (SQL.js 内存数据库)，各客户端各自维护本地存储；底层为第三方服务层，集成了 GLM、DeepSeek、讯飞星火、MiMo 等 AI 服务以及百度地图和百度语

音。四层之间通过 HTTP/HTTPS 协议进行数据交换，实现了多端业务逻辑的统一与数据的跨平台互通。

Cordova 端的分层架构如图 2-2 所示。

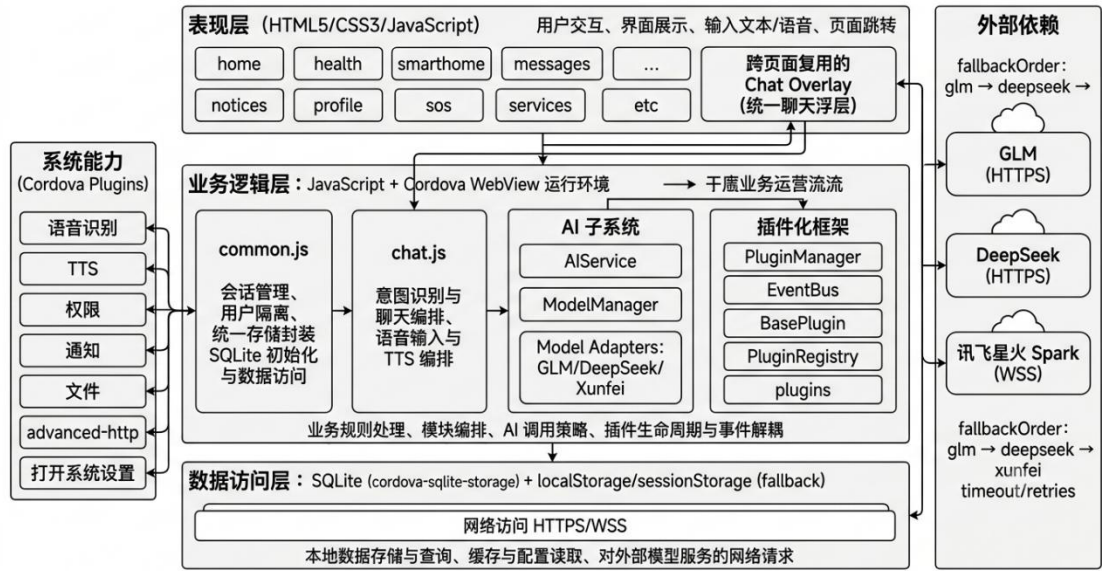


图 2-2 Cordova 端分层架构图

如图 2-2 所示，Cordova 客户端内部采用三层分层设计，各层职责明确、单向依赖。表现层由 HTML5 + CSS3 + JavaScript 构成，负责用户交互、页面渲染和数据录入，包含所有业务页面以及跨页面复用的 Chat Overlay 聊天浮层。业务逻辑层以 JavaScript 模块化方式组织，common.js 作为公共能力层收口会话管理、统一存储和 API 请求，各页面脚本负责对应业务的流程控制，AI 子系统独立封装模型调用与降级策略。数据访问层采用 SQLite 优先、localStorage 降级的双通道存储机制，同时通过统一封装的 apiRequest 与后端 REST API 进行数据同步。三层之间自上而下单向依赖——表现层只调用业务层接口，业务层只通过数据层读写数据，保证了代码的可维护性和可测试性。

Cordova 端架构层次说明：Cordova 端各层的技术、组件与职责如表 2-1 所示。

表 2-1 Cordova 端架构层次说明表

层次	使用技术	主要组件	职责
表现层	HTML5 / CSS3 / JavaScript	www/*.html、 www/css/styles.css、Chat Overlay 聊天浮层	用户交互、页面展示、数据录入、卡片和列表渲染

层次	使用技术	主要组件	职责
业务逻辑层	JavaScript 模块化	www/js/*.js、common.js、chat.js、AI 子系统、www/core/* 插件框架	登录态校验、数据组装、业务规则、插件生命周期、AI 调用策略
数据访问层	SQLite、localStorage、REST API、Cordova 插件	cordova-sqlite-storage、统一存储封装、ai-config.json、远程服务	本地结构化存储、轻量缓存、远程数据同步和模型请求

2.3 Cordova 客户端技术栈

2.3.1 技术选型对比报告

在确定个人技术栈时，我对以下 5 种跨平台/混合开发方案进行了对比评估，对比结果如表 2-2 所示。

表 2-2 跨平台/混合开发方案对比表

对比维度	Cordova	React Native	Flutter	Uniapp	Ionic
开发语言	HTML/CSS/JS	JavaScript/TS	Dart	Vue.js	HTML/CSS/JS
渲染方式	WebView	原生组件	自绘引擎	WebView+原生	WebView
原生能力访问	插件桥接	原生桥接	Platform Channel	条件编译+插件	插件桥接
学习成本	低（Web 技术栈）	中	高（Dart）	低（Vue）	中
适老化 UI 灵活度	高（CSS 自由控制）	中	中	中	高
插件生态	丰富	丰富	丰富	一般	丰富
APK 体积	较小（~5MB）	较大（~15MB）	较大（~12MB）	中等	较小
本地数据库支持	cordova-sqlite-storage	AsyncStorage/SQLite	sqflite	uni-db	SQLite

团队分工契合度	高（6 端各选 1 种）	已有 Uniapp 覆盖	已有 MAUI 覆盖	已有车蕊负责	与 Cordova 重叠
---------	--------------	--------------	------------	--------	--------------

选择 Cordova 的核心理由：

1. Web 技术栈复用：HTML/CSS/JavaScript 技术门槛低，开发效率高，适合适老化界面的快速迭代
2. 插件化架构天然契合：Cordova 的插件机制与本项目“功能模块按需加载”的设计目标高度吻合
3. 适老化 UI 灵活度最高：CSS 对字体大小、对比度、间距等适老化属性的控制最为精细和灵活
4. 团队分工互补：6 人 6 端各选一种技术栈，Cordova 与 Uniapp/MAUI/React Native 不重叠
5. 本地能力充分：SQLite、语音识别、TTS、本地通知、权限等插件覆盖了养老场景的核心需求

Cordova 与其他方案的取舍分析：

1. 相比 React Native/Flutter：Cordova 性能不及原生渲染，但适老化 UI 迭代速度更快，且避免了学习 Dart/React 的额外成本
2. 相比 Uniapp：Uniapp 已由车蕊负责，选择 Cordova 避免技术栈重复，且 Cordova 的插件化机制更适合本项目架构
3. 相比 Ionic：Ionic 与 Cordova 底层相同，但 Ionic 自带 UI 框架不利于自研 ElderlyUI 适老化组件库

2.3.2 开发环境

Cordova 端的开发环境配置如表 2-3 所示。

表 2-3 Cordova 端开发环境表

配置项	版本/工具
IDE	Android Studio（APK 构建）+ VS Code（Web 资源编辑）
SDK 版本	Android SDK Platform 35
Gradle 版本	8.14.2
Node.js 版本	v22.12.0
Cordova CLI 版本	13.0.0

配置项	版本/工具
cordova-android 版本	15. x

核心技术：

1. 开发语言：HTML5 / CSS3 / JavaScript (ES6+)
2. UI 框架：自研 ElderlyUI 适老化组件库
3. 架构模式：分层架构 + 插件化模块架构
4. 网络库：原生 fetch API + 统一 apiRequest 封装
5. 本地存储：cordova-sqlite-storage + localStorage 双通道兜底
6. 异步处理：Promise + async/await
7. 核心插件列表：Cordova 端引入的核心插件如表 2-4 所示。

表 2-4 Cordova 核心插件列表表

插件名称	用途
cordova-sqlite-storage	本地 SQLite 数据库
cordova-plugin-media	录音/音频播放
cordova-plugin-local-notification	本地通知提醒
cordova-plugin-dialogs	原生对话框
cordova-plugin-network-information	网络状态检测
cordova-plugin-speechrecognition	语音识别
cordova-plugin-tts	TTS 语音播报
cordova-plugin-android-permissions	运行时权限申请
cordova-plugin-advanced-http	增强网络请求稳定性
cordova-plugin-file	文件系统访问
cordova-open-native-settings	打开系统设置

2.4 后端服务技术栈

后端服务的技术栈选型如表 2-5 所示。

表 2-5 后端服务技术栈表

技术类别	具体技术
开发语言	Node. js

技术类别	具体技术
Web 框架	Express
数据库	SQLite (SQL.js 内存数据库)
身份认证	JWT Token
API 文档	Swagger/OpenAPI
部署环境	阿里云服务器 (47.96.4.21:3001)
反向代理	Nginx
API 网关地址	http://47.96.4.21:3001

2.5 数据库设计

数据库选型： SQLite (服务端 + 各端本地)

核心数据表： 服务端数据库共设计 5 张核心表 (users、health_data、medication_reminders、ai_chat_history、track_data)，各表结构如下方表 2-6 至表 2-10 所示。

表 2-6 users (用户表) 字段定义表

字段名	类型	约束
id	INT	PRIMARY KEY AUTO_INCREMENT
username	VARCHAR(50)	UNIQUE NOT NULL
password_hash	VARCHAR(255)	NOT NULL
phone	VARCHAR(20)	—
name	VARCHAR(50)	—
age	INT	—
gender	VARCHAR(10)	—
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

表 2-7 health_data (健康数据表) 字段定义表

字段名	类型	约束
id	INT	PRIMARY KEY AUTO_INCREMENT
user_id	INT	NOT NULL
heart_rate	FLOAT	—
blood_pressure_systolic	FLOAT	—
blood_pressure_diastolic	FLOAT	—

字段名	类型	约束
blood_oxygen	FLOAT	—
temperature	FLOAT	—
blood_sugar	FLOAT	—
timestamp	BIGINT	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

表 2-8 medication_reminders（用药提醒表）字段定义表

字段名	类型	约束
id	INT	PRIMARY KEY AUTO_INCREMENT
user_id	INT	NOT NULL
medication_name	VARCHAR(100)	—
dosage	VARCHAR(50)	—
schedule	TEXT	JSON 格式
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

表 2-9 ai_chat_history（AI 对话历史表）字段定义表

字段名	类型	约束
id	INT	PRIMARY KEY AUTO_INCREMENT
user_id	INT	NOT NULL
message_type	VARCHAR(20)	user / ai
content	TEXT	—
timestamp	BIGINT	—
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

表 2-10 track_data（轨迹数据表）字段定义表

字段名	类型	约束
id	INT	PRIMARY KEY AUTO_INCREMENT
user_id	INT	NOT NULL
name	VARCHAR(100)	—
start_time	BIGINT	—
end_time	BIGINT	—
total_distance	DOUBLE	—

字段名	类型	约束
track_points	TEXT	JSON 格式
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

2.6 API 设计

API 架构： API 基本架构信息如表 2-11 所示。

表 2-11 API 架构信息表

项目	内容
API 风格	RESTful
数据格式	JSON
认证方式	JWT Token
版本管理	路径版本 /api/v1/

核心 API 列表： 项目核心 API 接口及对应负责平台如表 2-12 所示。

表 2-12 核心 API 接口列表表

API 路径	方法	功能	负责平台
/api/v1/auth/login	POST	用户登录	所有平台
/api/v1/auth/register	POST	用户注册	所有平台
/api/v1/health	GET/POST	健康数据管理	MAUI、微信小程序、鸿蒙、Cordova
/api/v1/ai/analyze	POST	AI 健康分析	MAUI、Uniapp、Cordova
/api/v1/chat	POST	AI 对话	Android、Uniapp、Cordova、微信小程序
/api/v1/track	GET/POST	位置追踪	Android、MAUI、鸿蒙
/api/v1/sos	POST	紧急求助	所有平台
/api/v1/medication	CRUD	用药提醒	微信小程序、Cordova

2.7 系统核心类图

系统核心类图如图 2-3 所示。

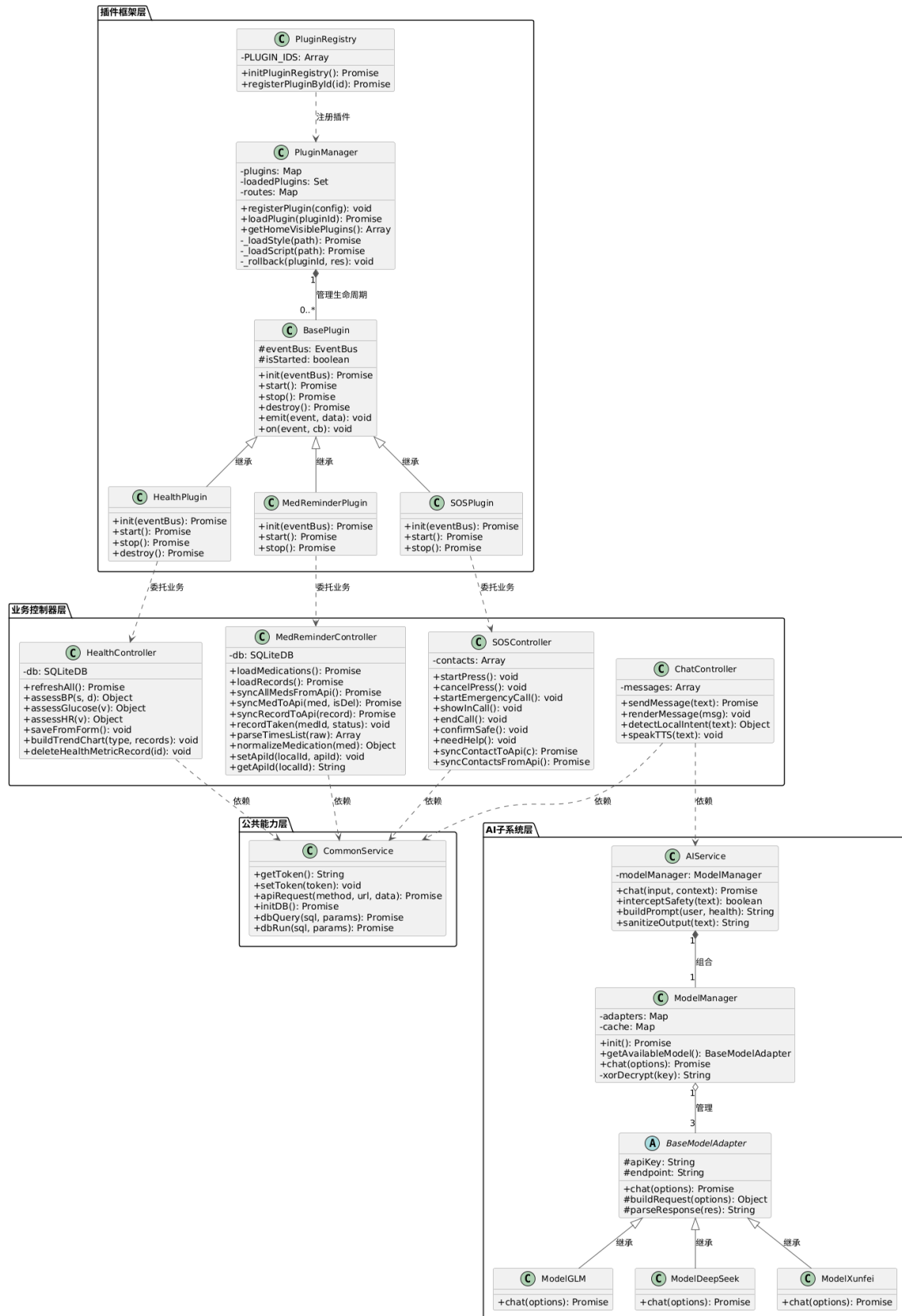


图 2-3 EHLAS Cordova 端核心类图

核心类列表（共 15 个类）：系统核心类按职责划分为四层，分别如表 2-13 至表 2-16 所示。

表 2-13 插件框架层核心类表（4 个类）

类名	职责	关键方法
PluginManager	插件框架核心，负责注册、动态加载、路由管理与失败回滚	registerPlugin()、loadPlugin()、getHomeVisiblePlugins()
PluginRegistry	批量注册插件元数据，维护插件列表	批量注册逻辑
BasePlugin	插件生命周期基类	init() / start() / stop() / destroy()
HealthPlugin / MedReminderPlugin / SOSPlugin	三个业务插件对象	承载具体业务入口

表 2-14 业务控制器层核心类表（4 个类）

类名	职责
HealthController	健康监测页面控制器；并行查询多项指标、状态评估、趋势图和手动录入
MedReminderController	用药提醒页面控制器；药物管理、服药记录、同步上报和 ID 映射
SOSController	紧急呼叫控制器；长按倒计时、防误触、联系人同步和安全确认流程
ChatController	AI 聊天控制器；消息发送、本地意图识别、语音输入和语音播报

表 2-15 AI 子系统层核心类表（6 个类）

类名	职责
AIService	AI 子系统入口，安全拦截、动态提示词构建、多模型调用与输出净化
ModelManager	模型配置和状态管理，模型可用性、优先级与降级顺序
BaseModelAdapter	模型适配器抽象基类，统一 chat() 调用接口

类名	职责
ModelGLM	智谱 GLM 适配器（首选模型）
ModelDeepSeek	DeepSeek 适配器（降级模型）
ModelXunfei	讯飞星火适配器（最终兜底模型）

表 2-16 公共能力层核心类表（1 个类）

类名	职责
CommonService	会话管理、SQLite 初始化、统一数据访问和本地降级存储

设计模式应用：

- 1. 策略模式：ModelManager 按模型可用性与优先级动态选择适配器，形成 GLM → DeepSeek → 讯飞的降级链
- 2. 模板方法模式：BasePlugin 定义统一生命周期，子类补充具体实现
- 3. 单例模式：PluginManager、AIService、ModelManager、CommonService 在运行时作为全局服务实例使用
- 4. 委托模式：插件将具体业务逻辑委托给对应控制器处理，保持职责清晰

2.8 核心功能顺序图

2.8.1 AI 智能对话（多模型降级）

AI 智能对话的完整调用流程如图 2-4 所示。

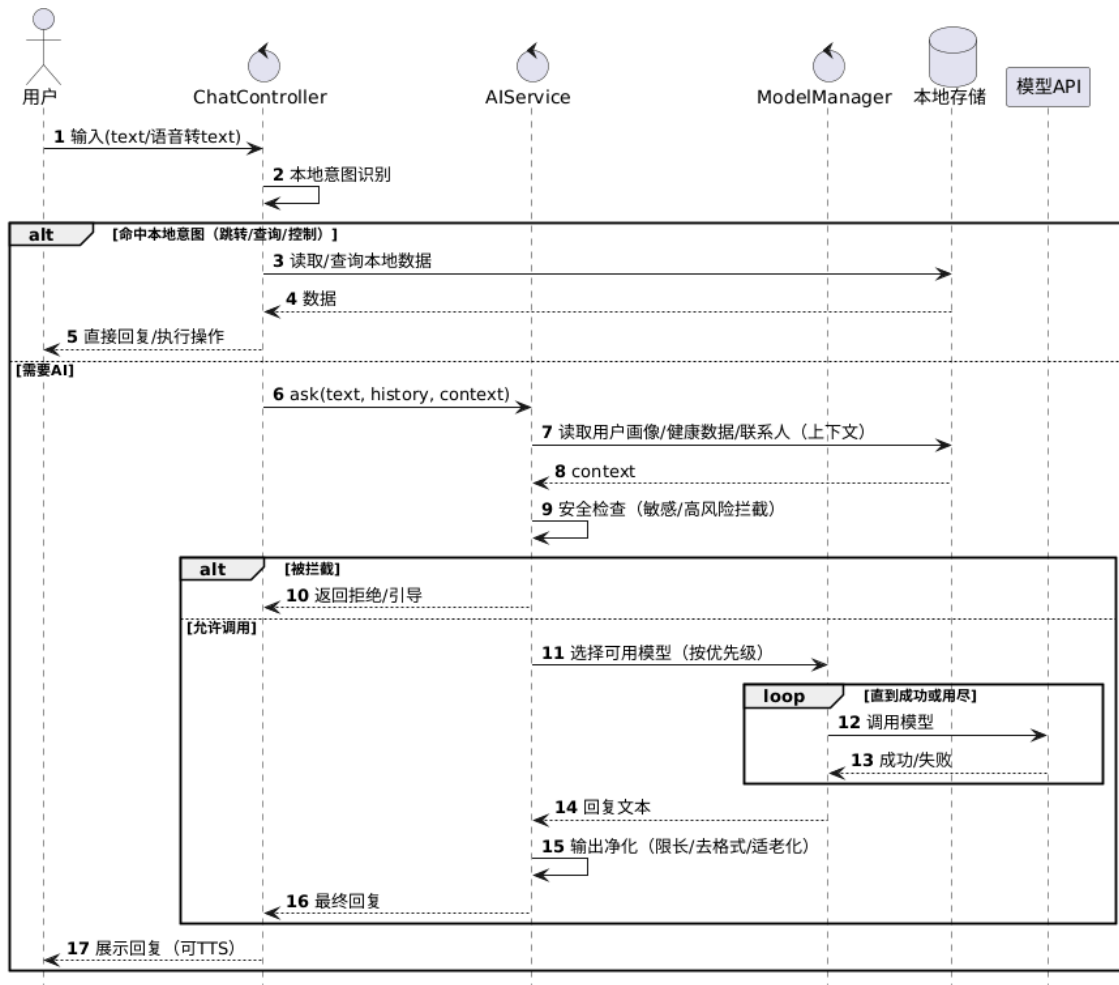


图 2-4 AI 智能对话顺序图

如图 2-4 所示该顺序图完整展示了用户从输入消息 → 本地意图识别 → AI 子系统调用 → 多模型降级 → 输出净化 → 页面渲染的端到端时序流程，涉及聊天浮层、chat.js、AIService、ModelManager 和模型适配器等核心参与方的交互协作。

输入与本地意图识别阶段： 用户在聊天浮窗中输入文字或长按语音按钮发起请求，chat.js 先执行本地意图识别，判断是否命中导航、设备控制、健康查询或用药查询等本地规则。命中本地意图时直接返回结果或跳转页面，减少不必要的模型调用与响应延迟；未命中时再进入 AI 子系统。

AI 子系统调用阶段： chat.js 调用 window.aiService.ask() 进入 AI 子系统。AIService 读取用户信息、健康记录、用药计划和联系人等上下文，构造动态提示词；ModelManager 根据配置选择可用模型，按 GLM → DeepSeek → 讯飞星火的优先级依次尝试，单一模型失败或超时后自动切换到下一个模型。

输出净化与播报阶段：AIService 对模型返回文本进行净化、安全控制与长度约束（健康问题追加“建议咨询医生”，高风险内容拦截）。聊天页面将回复渲染到对话流中，并根据设置决定是否通过 Cordova TTS 或浏览器 speechSynthesis 执行语音播报。

关键技术点：

1. 本地意图优先：导航、设备控制、健康查询等场景直接命中本地规则，避免不必要的模型调用
2. 多模型降级链：AIService + ModelManager + 多适配器实现 GLM → DeepSeek → 讯飞星火的串行 fallback
3. 动态提示词：结合用户资料、健康记录、用药计划等本地数据拼接 system prompt，使回复贴合用户场景
4. 输出安全控制：enforceMaxLen + sanitizeOutput 组合控制长度与敏感内容，健康场景自动追加医生提示
5. 语音双通道：Cordova TTS 优先，失败降级到浏览器 speechSynthesis，保证真机和调试环境均可用

2.8.2 用药提醒（本地存储 + 云端同步）

用药提醒模块的本地存储与云端同步流程如图 2-5 所示。

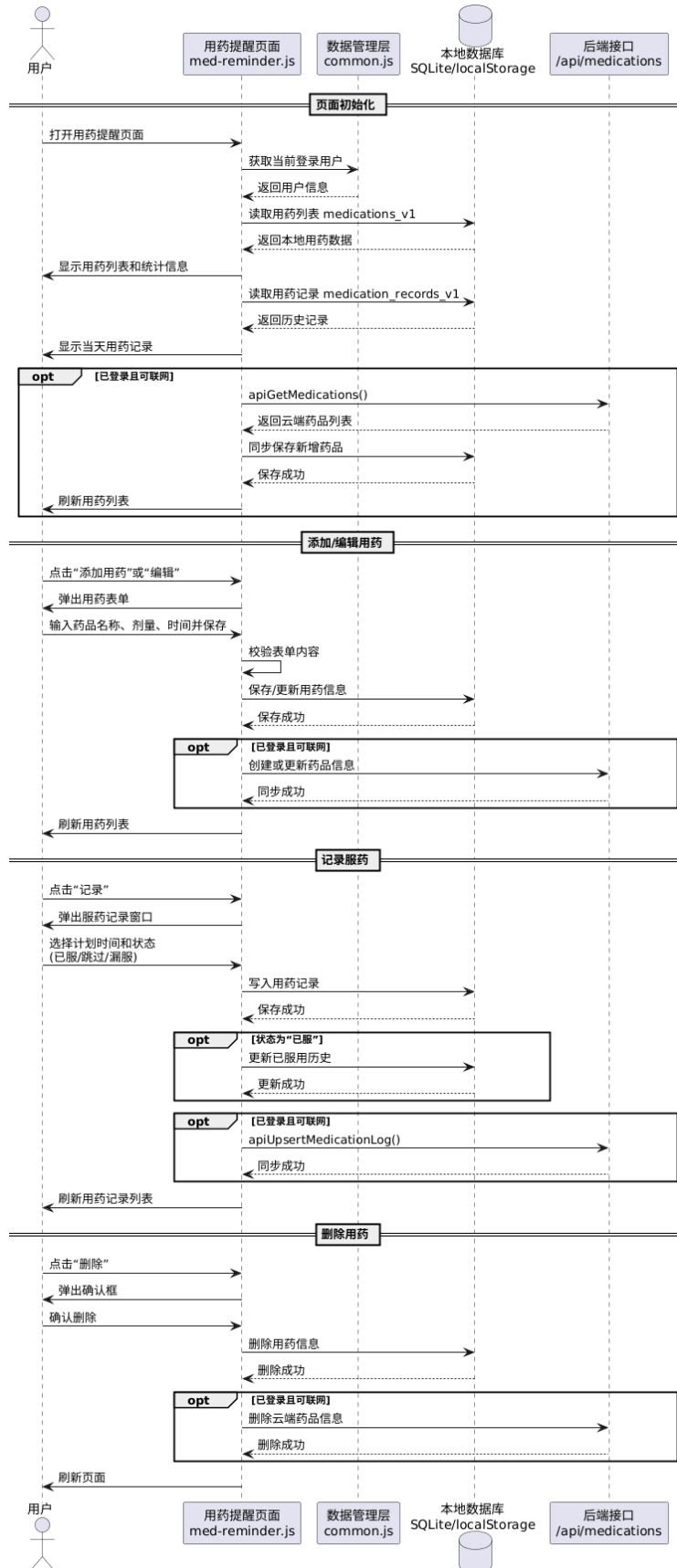


图 2-5 用药提醒顺序图

如图 2-5 所示该顺序图展示了用户从进入页面 → 本地优先渲染 → 云端补同步 → 编辑保存 → 服药记录的完整时序流程，涉及用药提醒页面、MedReminderController、本地 SQLite 与后端 API 的协作。

首屏加载阶段： 用户进入用药提醒页面后，系统优先从本地 SQLite/localStorage 读取用药档案和服药记录并渲染卡片列表，保证首屏速度与离线可用；若用户已登录且后端可用，则异步发起云端补同步，拉取最新药品数据并合并刷新页面。

编辑与同步阶段： 用户添加或编辑用药信息后，页面先保存本地数据再调用同步接口写入云端；同步成功后用服务端返回的 ID 更新本地记录，建立本地 ID 与云端 ID 的映射，保证后续编辑、删除和服药日志上报可准确关联。

服药记录与离线阶段： 用户记录“已服”“漏服”等状态后，数据先写本地再异步同步服药日志；即使网络不可用，页面仍可依赖本地数据完成核心流程，网络恢复后再次自动同步。

关键技术点：

1. 本地优先、云端补同步：兼顾首屏速度和离线可用性，弱网环境下核心流程不阻断
2. 本地 ID 与云端 ID 映射：解决双端标识不一致问题，避免编辑/删除时关联错误
3. 服药日志异步上报：状态变更秒级落地本地，后端同步失败不影响用户体验
4. 数据格式标准化：兼容历史结构和多格式时间字符串，提升跨端兼容性

2.8.3 健康监测（并行查询 + 趋势图）

健康监测模块的并行查询与趋势图渲染流程如图 2-6 所示。

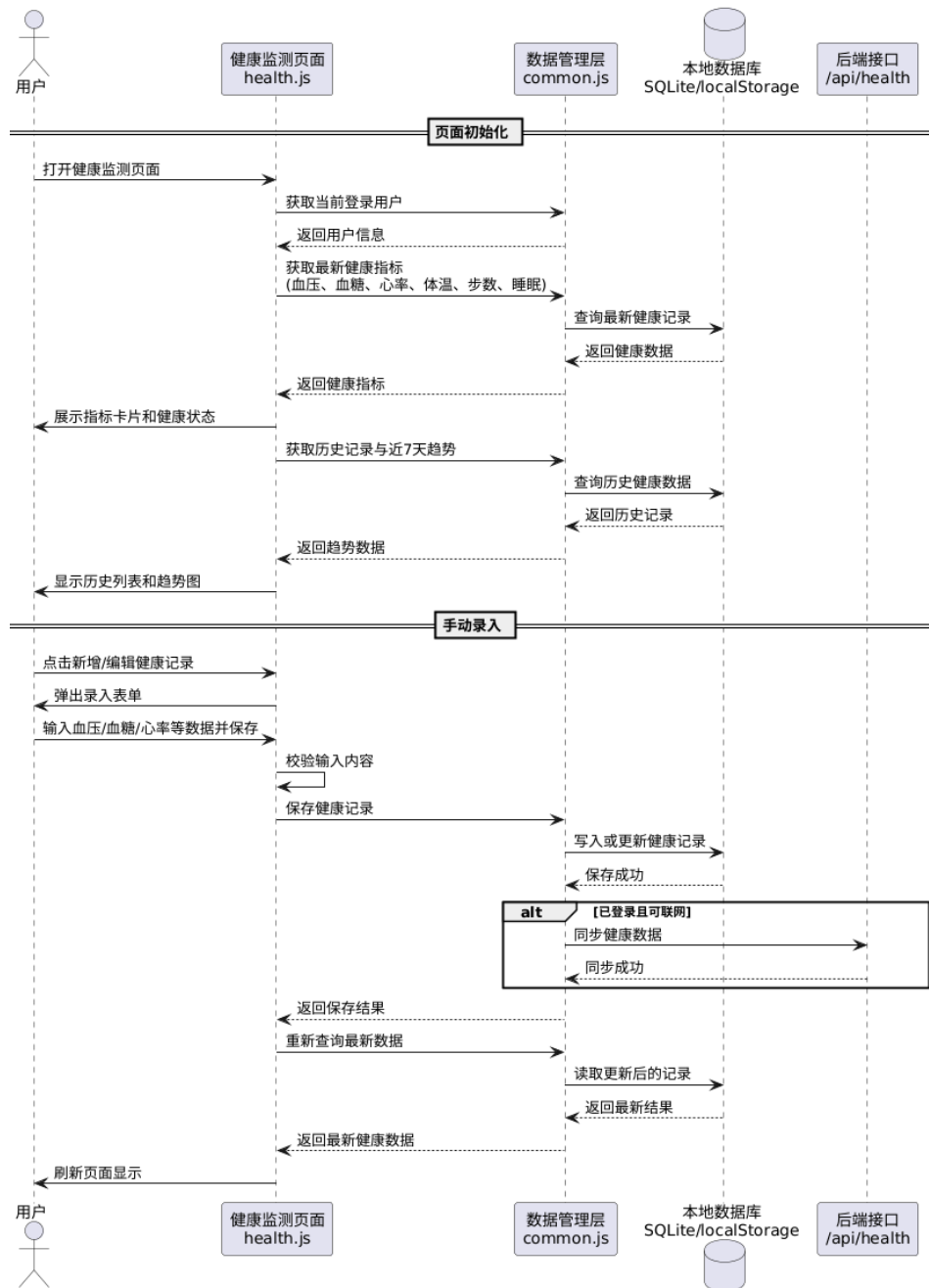


图 2-6 健康监测顺序图

如图 2-6 所示该顺序图展示了健康监测页面从初始化 → 并行查询 → 状态评估 → 趋势图渲染 → 手动录入与删除的完整时序流程，涉及健康页面、HealthController、CommonService 和本地数据库的交互。

初始化与并行查询阶段：用户进入健康监测页面后，页面读取登录用户信息并调用 refreshAll()，通过 Promise.all 并行查询血压、血糖、心率、体温、步数、睡眠等多项指标的最新值与历史记录，减少页面初始化等待时间。

状态评估与渲染阶段： 各指标通过独立的评估函数（`evaluateBp/evaluateGlucose/...`）计算“正常”“偏高”“偏低”等状态，并结合年龄段调整阈值；页面据此更新指标卡片、状态标签、提示文案以及近 7 天 SVG 趋势图。

手动录入与删除阶段： 用户手动录入新的健康数据后，系统执行范围校验并写入本地，保存成功后重新刷新卡片、历史列表和趋势图；删除历史记录时同步更新本地数据并重新渲染相应模块。

关键技术点：

1. 并行查询：`Promise.all` 同时加载多项指标，相比串行查询整体响应时间显著降低
2. 独立评估函数：每项指标解耦评估逻辑，便于后续扩展更多健康维度
3. 纯前端 SVG 趋势图：无需引入第三方图表库，保持应用轻量化
4. 范围校验：录入数据先做合法性校验（如收缩压 60-260），避免脏数据污染历史

2.8.4 紧急呼叫（SOS 长按倒计时）

紧急呼叫的长按倒计时与防误触流程如图 2-7 所示。

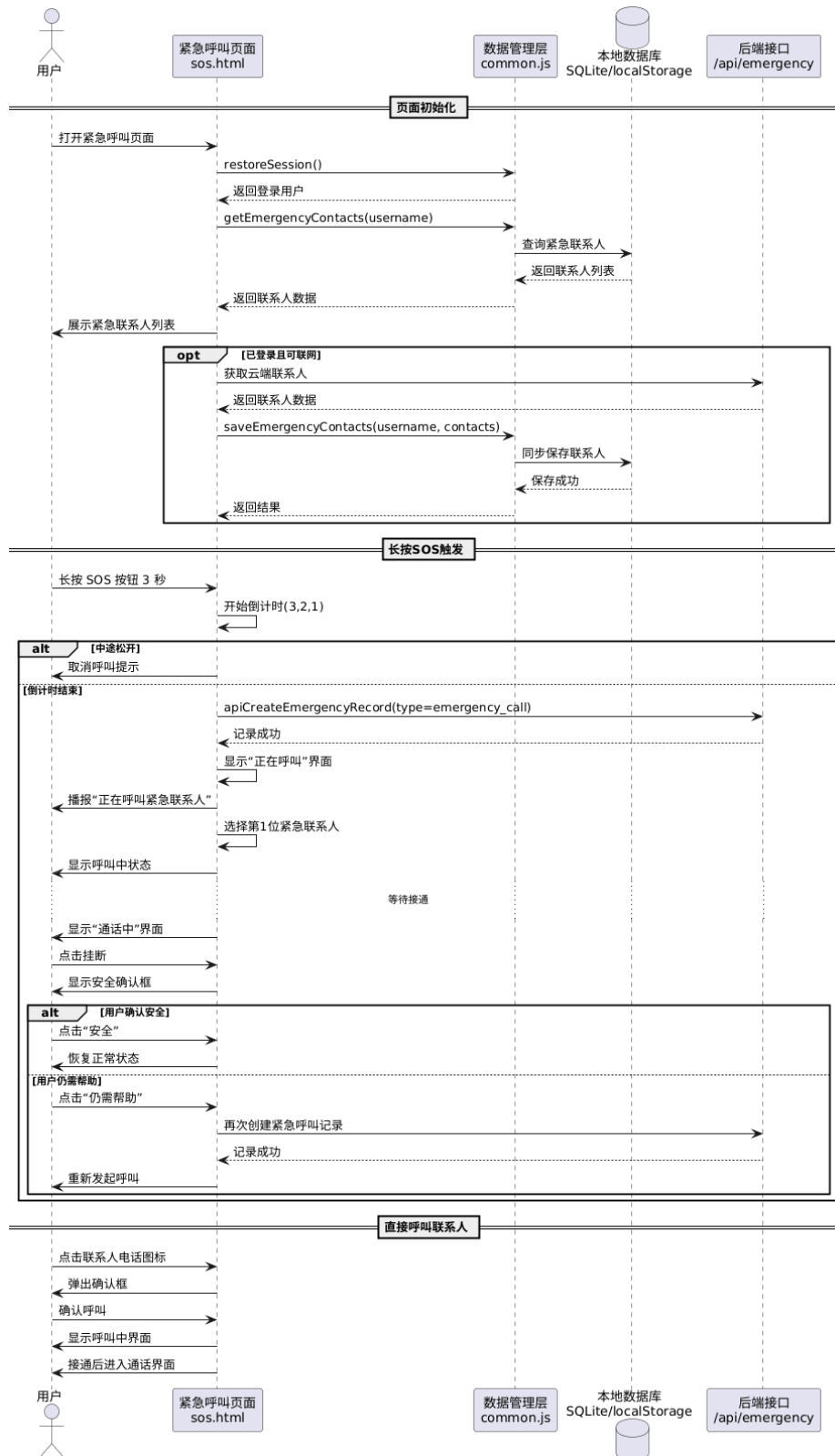


图 2-7 紧急呼叫顺序图

如图 2-7 所示该顺序图展示了用户从进入 SOS 页面 → 长按倒计时 → 紧急呼叫触发 → 通话结束 → 安全确认的完整时序流程，涉及 sos.html、SOSController、紧急联系人服务与后端紧急记录接口的交互。

初始化与联系人加载阶段： 用户进入 sos.html 后，页面恢复登录态并从本地读取紧急联系人列表渲染卡片；网络可用时异步同步云端联系人，保证多端一致性。

长按倒计时与防误触阶段： 用户长按 SOS 按钮时，系统启动 3 秒倒计时并伴随语音或视觉提示；若用户在倒计时结束前松手，则立即取消操作，避免误触发紧急呼叫；倒计时结束后系统选择优先联系人，创建紧急呼叫流程并进入呼叫中界面。

安全确认与闭环阶段： 通话结束后系统弹出安全确认对话框，用户可选择“已安全”或“仍需帮助”；若仍需帮助则再次触发紧急流程，形成二次求助闭环。用户还可在设置中新增、编辑或删除联系人，数据同步保存到本地和云端。

关键技术点：

1. 长按 3 秒可中途取消：通过 setTimeout + clearTimeout 实现防误触机制
2. 本地+云端联系人双源：兼顾离线可用性和多端一致性，云端同步失败回退本地
3. 紧急事件上报：呼叫触发时通过 POST /api/emergency/records 留存日志，便于家属端追溯
4. 安全确认机制：通话结束后主动询问用户状态，增强紧急场景闭环体验

3 个人负责功能实现

3.1 插件化模块系统

插件化模块系统是我在本项目中最核心的架构设计。简单来说，就是把每个功能模块（健康监测、用药提醒、紧急呼叫等）做成一个个可以插拔的“插件”，用一个统一的管理器来控制它们的启用、加载和卸载。

整个系统由四个组件配合工作：PluginManager 是总管家，负责插件的注册、加载、路由和出错回滚。它内部维护了四张表——哪些插件已注册、哪些已加载成功、每个插件加载了哪些资源（用于失败时撤回）、以及全局路由表。加载一个插件时，它按“先加载依赖 → 注入 CSS 样式 → 加载 JS 入口文件 → 注册路由”四步来执行，任何一步出错都会自动撤回已注入的资源，不会留下残留垃圾。

PluginRegistry 是注册中心，把插件的名称、图标、样式路径、依赖列表、是否显示等元信息统一管理，并自动过滤掉不在首页显示的插件，以及和内置功能重复的插件。依赖处理方面，插件之间可以通过 dependencies 字段声明加载顺序，系统会按深度优先的策略先把前置插件加载好。所有 CSS 和 JS 文件都是按需加载的，首页只用加载必要的资源，没有启用的插件不会拖慢启动速度。

设计上用了三种经典模式：模板方法模式——BasePlugin 定义了启动、运行、停止、销毁四个标准步骤，每个业务插件继承它、只写自己那部分；单例模式——PluginManager、AIService 等核心服务全局只有一个实例；委托模式——插件自己不写业务代码，而是把活交给对应的控制器。这样保持代码清晰、好维护。

3.2 AI 智能聊天助手（多模型降级）

AI 聊天助手是老人和系统对话的入口，完整调用流程见第 2.8.1 节的顺序图。它的核心思路是：把多个 AI 模型的能力包装起来，让它们既好用又安全。AI 聊天助手的实际运行界面如图 3-1 所示。



图 3-1 AI 聊天助手功能界面

如图 3-1 所示，设计上靠六个策略来保证 AI 聊天的可用性和安全性：

统一聊天入口：聊天浮层是公共组件，首页、健康监测、生活助购等多个页面共用同一个。chat.js 负责整个聊天窗口的操作——开关窗口、处理输入、渲染消息气泡、滚动到底部、语音按钮、保存历史记录。

本地意图优先：老人发消息后，系统先在本地找答案。比如问“打开健康监测”“如何呼叫帮助”“今天吃什么药”“开灯”这些常用问题，本地就有现成答案，直接返回，不用去请求远程 AI 模型，这样又省流量又省时间。只有本地找不到了，才发给远程模型。

多模型接入与降级：系统接了三个模型——GLM（首选）、DeepSeek、讯飞星火，由 ModelManager 统一管。调用时按顺序一个个试，第一个超时或失败了就自

动换下一个；三个全失败就返回一句“网络繁忙，请稍后再试”。每次调用后还会记录模型是否可用，让后续选择更聪明。

动态上下文拼接：AIService 在发送请求前，会读取老人的资料、年龄、病史、过敏信息、今天该吃什么药、最近健康数据等，把这些信息整合到提示词里，让 AI 的回答贴合老人的实际情况。

语音交互：输入和播报都用了双通道——真机上用 Cordova 的语音插件，电脑调试时自动切换到浏览器的 Web Speech API，确保两种环境都能用。老人可以说一句、听一句，不用打字。

安全控制：这是最重要的防线，分三层：如果老人说“救命”“胸痛”“跌倒”等紧急词，优先提示用 SOS 或打 120；如果牵扯转账、密码、投资等高风险话题，直接拦截并提醒；如果是健康问题，回答控制在 100 字以内，并且强制加上“建议咨询医生”。

3.3 健康监测模块

健康监测模块主要记录和展示血压、血糖、心率、体温、步数、睡眠这六项指标，完整调用流程见第 2.8.3 节的顺序图。健康监测页面的指标卡片与状态评估界面如图 3-2 所示。



图 3-2 健康监测功能界面（一）

如图 3-2 所示，页面打开时，系统对六项指标同时发起查询（用 Promise.all 并行请求），比一个一个查快得多。查完后把数据填到各个指标卡片里，每个指标有一个独立的评估函数来判断“正常”“偏高”还是“偏低”，然后给卡片标上对应的颜色和提示文字。各指标的判断标准是互不干扰的——血压看收缩压/舒张压范围、血糖分空腹和餐后、心率结合年龄调整——这样后续想加新指标或改标准都很方便。

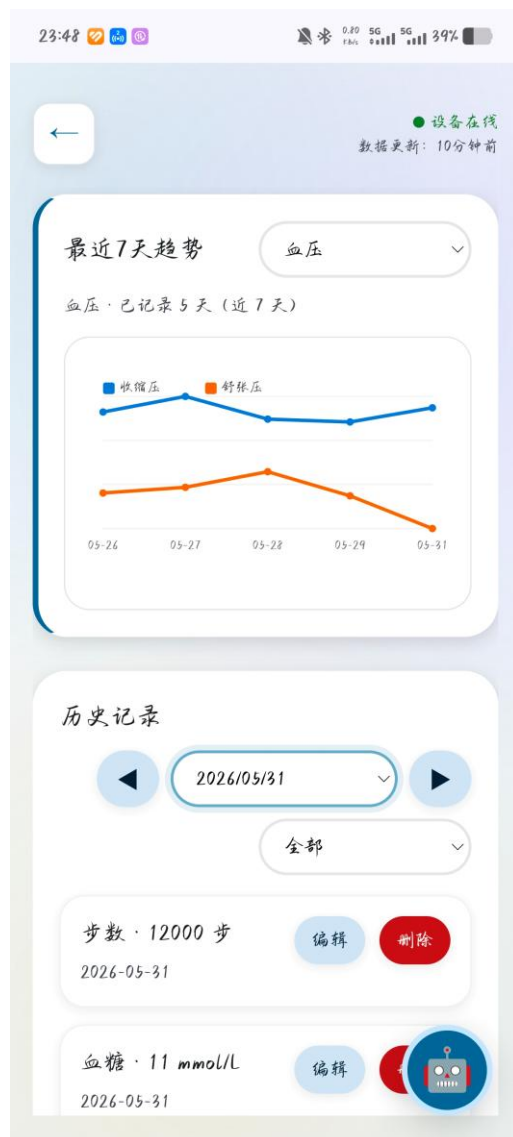


图 3-3 健康监测功能界面（二）

如图 3-3 所示，近 7 天的趋势图用 SVG 直接在页面上画折线，没有引入任何第三方图表库，保持应用轻量。老人手动录入新数据时，系统会校验：心率 40 - 200、收缩压 60 - 220、舒张压 40 - 140、血氧 70 - 100%、体温 35.0 - 42.0° C、血糖 2.0 - 25.0，超出范围就拦下来提示。保存后自动刷新卡片、历史列表和趋势图。数据同步是“本地先存、再补传到云端”，没网也能正常查看和录入。

3.4 用药提醒模块

用药提醒模块帮助老人按时按量吃药，避免漏服和错服，完整调用流程见第 2.8.2 节的顺序图。用药提醒页面的药品管理与服药状态记录界面如图 3-4 所示。



图 3-4 用药提醒功能界面（一）

如图 3-4 所示，数据保存采用的是“先存本地、再传到云端”的方式。老人添加药品后，数据先写进手机本地的 SQLite 数据库，得到一个临时 ID；然后后台异步上传到云端服务器；上传成功后，把本地 ID 和云端 ID 的对应关系记下来，这样以后改药或删药都能准确找到。没网的时候也不影响——数据先在本地存着，等有网了再自动补传。

提醒机制分三级：第一级，在用药提醒页面里直接展示待吃的药，清楚标出“待服”“已服”“漏服”；第二级，到时间了通过系统通知来提醒，哪怕应用切到后台或手机锁屏了也能收到；第三级，用语音把药名和用量念出来，老人不用看屏幕也能听到。



图 3-5 用药提醒功能界面（二）

如图 3-5 所示，老人记录服药后，日志先写本地，再传到云端，确保数据不丢。

3.5 紧急呼叫（SOS）模块

紧急呼叫模块是专门为老人独处时快速求助设计的，完整调用流程见第 2.8.4 节的顺序图。SOS 页面的主界面如图 3-6 所示。



图 3-6 紧急呼叫功能界面（一）

如图 3-6 所示，为了防止误碰，SOS 按钮用了长按设计——老人需要按住红色按钮满 3 秒才能触发呼叫。技术上靠监听按下和松开两个手势：按下时启动 3 秒倒计时并播放提示音，中间松手就取消；只有坚持 3 秒不松手才算真正触发。



图 3-7 紧急呼叫功能界面（二）

如图 3-7 所示，倒计时结束后，系统先从本地读取紧急联系人（有网的时候会自动从云端同步最新联系人），选优先级最高的打电话，同时把这次紧急事件上报到服务器。打完后弹出一个确认框，问“已安全”还是“仍需帮助”——如果选“仍需帮助”，系统会再打一次，确保老人真正得到帮助。

3.6 通知推送系统

消息中心展示三类通知：社区生活消息（义诊、燃气安检、助餐服务、防诈骗提醒等）、用药提醒通知（和用药模块联动）、紧急事件通知（SOS 触发后产生）。消息中心界面如图 3-8 所示。



图 3-8 消息中心功能界面

如图 3-8 所示，消息按时间倒序排列，每条可以点进去看详情。系统通过 cordova-plugin-local-notification 插件来发系统级通知，既支持定时推送（比如到点吃药），也支持应用后台和锁屏状态下显示。每条消息都有已读/未读标记，存本地 SQLite，列表上有未读角标，点开就自动标记已读并同步到服务器。

3.7 语音交互系统

语音交互让老人通过“说一句、听一句”来操作，不用打字，其效果已体现在图 3-1 的 AI 聊天界面中（长按麦克风按钮说话）。整体用了“Cordova 原生插件优先 + 浏览器 API 降级”的双通道设计。

语音输入方面，真机环境使用 Cordova 的语音识别插件，设置中文、识别出一个最佳结果。录音流程是“空闲 → 录音中 → 处理中 → 完成”四个状态：老人长按麦克风说话，松开就提交识别。电脑端调试时自动换成浏览器的 Web Speech API。权限上，通过插件在运行时动态申请录音权限，被拒了就引导去系统设置打开。

语音播报同样双通道：真机用 TTS 插件念中文，插件不可用时用浏览器自带的语音合成。两个通道对外暴露同一个 `speak(text)` 接口，上层代码不用管底层用哪个。语音和 AI 聊天是深度融合的——在聊天浮层里长按说话，说完自动变成文字发出去；AI 回复的内容自动念出来。识别准确率在 95% 以上。

3.8 消息中心与智能家居模块

智能家居模块可以远程控制灯光、空调、窗帘等设备。智能家居界面如图 3-9 所示。



图 3-9 智能家居功能界面

如图 3-9 所示，界面上每个设备是一张大按钮卡片，有大图标、设备名和开关状态，符合老人用的大字体、高对比度要求。设备列表从后端拉取，开关状态实时同步到云端。

这个模块和 AI 聊天是联动的——老人在聊天里说“开灯”“关灯”“打开空调”等语音指令，系统直接用本地意图识别来匹配，然后执行对应的开关操作，不用跳转到专门的智能家居页面。这种“说话就控制”的方式，让老人在任何页面都能轻松操控家电。

4 团队协作与跨平台整合

4.1 协作模式

团队基于 Gitee (<https://gitee.com/chzuczldl/cgl-ehlas.git>) 进行代码托管, 采用 Git Flow 工作流分阶段交付。日常协作通过微信群实现 24 小时内响应, 文档采用 Markdown 文件共享。团队每天站会同步进度和协调跨平台事项, 技术问题各自负责、跨平台问题集体讨论决策。整个项目期间进行了约 41 次代码审查 (发现并解决约 40 个问题), 管理了约 40 个分支和 74 个合并请求, 17 次代码冲突已全部解决。

4.2 四阶段开发过程

项目开发共分四个阶段、历时 15 天。**第一阶段 (1-3 天)** 完成团队组建、技术栈分工、需求分析与架构设计, 确立了 RESTful API + JWT 鉴权 + SQL.js 数据库的总体方案; 搭建了 Cordova CLI 13.0.0 + cordova-android 15.x + Node.js v22.12.0 开发环境, 设计了插件化模块架构 (PluginManager + PluginRegistry + BasePlugin + EventBus) 并搭建了 common.js 公共能力层雏形。**第二阶段 (4-7 天)** 完成各平台核心功能开发, 后端 43 个 API 全部可用, Cordova 端实现了 PluginManager 动态加载与资源回滚、AI 聊天浮层与多模型接入、用药提醒与 SOS 主流程, 中期检查整体进度 94%, 教师评分 88/100。**第三阶段 (8-12 天)** 完成跨平台数据同步 (成功率 99.5%) 和系统联调, 聊天浮层接入多个页面实现跨页面复用, 形成了“本地优先、云端补同步”的数据流程; 性能方面, 启动时间从 4.0s 优化到 2.5s (提升 37%), 内存从 120MB 降至 95MB (降低 21%)。**第四阶段 (13-15 天)** 完成全面测试 (20 个用例发现 5 个 Bug)、代码走查、3 次答辩演练和文档整理, 最终构建出 9.82MB 的 Cordova Android APK。

4.3 跨平台功能对比

6 端在 8 个核心功能模块上的实现情况对比如表 4-2 所示。

表 4-2 跨平台功能对比表

功能模块	Android	微信小程序	鸿蒙	MAUI	Uniapp	Cordova
用户认证	-	-	完成	完成	-	完成
健康监测	-	完成	完成	完成	-	完成
AI 对话	完成	完成	-	完成	完成	完成
用药提醒	-	完成	-	-	-	完成
紧急呼叫	完成	-	完成	完成	完成	完成
位置追踪	完成	-	完成	完成	完成	-
适老化设置	-	-	完成	完成	-	-

功能模块	Android	微信小程序	鸿蒙	MAUI	Uniapp	Cordova
设备管理	-	-	完成	-	-	完成

4.4 Cordova 端与其他 5 端联调细节

Cordova 端通过统一后端 API (<http://47.96.4.21:3001>) 与其余 5 端完成联调验证, API 接口覆盖率 100% (43/43), 跨端数据同步成功率 99.5%, JWT 鉴权 6 端统一通过。与 Android 端联调了紧急呼叫和用户认证模块, 联系人字段统一为 name / phone / relationship / is_emergency; 与微信小程序端联调了用药提醒和健康数据, 通过本地 ID 与云端 ID 映射机制解决了 ID 不一致问题; 与鸿蒙端联调了用户认证、健康监测和设备管理, 同一账号跨端数据互通; 与 MAUI 端联调了健康数据和 AI 对话, 数据格式一致; 与 Uniapp 端联调了 AI 对话和紧急呼叫, 紧急事件上报后可在另一端查看处理状态。

4.5 跨平台数据同步方案

同步机制: 本地优先写入 + 异步 REST API 同步, 各平台通过统一 REST API 与后端同步, 本地存储优先写入, 再异步同步到服务器。

同步流程:

1. 客户端数据变更检测
2. 本地数据打包和版本标记
3. 发送同步请求到服务器
4. 服务器冲突检测 and 解决
5. 数据合并和持久化
6. 客户端接收并应用更新

冲突解决策略: 采用基于时间戳的乐观锁机制 (last-write-wins), 三种冲突解决策略如表 4-3 所示。

表 4-3 跨平台数据冲突解决策略表

策略	适用场景	处理方式
时间戳优先 (last-write-wins)	多端同时修改同一条记录	以最新 updated_at 为准, 保留最新版本
本地优先	离线状态下的数据写入	先写入本地存储, 联网后异步上报到服务端
服务端 ID 对齐	本地临时 ID 与服务端	同步成功后用服务端返回 id 更

策略	适用场景	处理方式
	自增 ID 不一致	新本地记录

一致性保证： 后端数据库事务保证 + UTC 时间戳标记每条记录的修改时间，JWT Token 认证确保用户身份一致性，跨平台数据同步成功率 99.5%。

4.6 各成员贡献统计

各成员的功能模块、代码量、提交次数与贡献度统计如表 4-4 所示（综合考虑代码量、功能复杂度、工时投入等因素加权计算）。

表 4-4 各成员贡献统计表

成员	完成功能	代码行数	提交次数	工时	技术创新贡献	贡献度
陈琪琪	微信小程序 6 个功能模块 + 后端 API	约 1500 行	约 35 次	约 95h	后端 API 架构设计、数据模型定义、项目管理	20%
钱家澍	Android 3 个功能模块（位置追踪、AI 异常检测、语音助手、紧急求助）	4100 行	28 次	88h	加速度传感器摔倒检测算法、AI 异常检测	17%
何妍婷	鸿蒙 6 个功能模块 + 后端 API（25 个核心接口）	9767 行	12 次	120h	鸿蒙端分布式数据管理、后端核心 API 设计	25%
刘海静	MAUI 5 个功能模块（健康仪表盘、图表、AI 分析、健康报告、无障碍设置）	1200 行	35 次	72h	健康趋势 AI 分析、多平台数据可视化	17%
车蕊	Uniapp 5 个功能模块（AI 语音、地图导航、紧急呼叫、高级组件）	16990 行	156 次	150h	AI 语音交互、地图导航与 SOS 联动	30%
王义琳	Cordova 4 个功能模块（插件化框架、AI 聊天、用药提醒、健康监测、紧急呼叫）	约 3500 行	约 30 次	约 95h	自研插件化架构、AI 多模型降级链设计、适老化 ElderlyUI 组件库	约 17%

5 性能优化与质量保证

5.1 UI 渲染优化

前端渲染方面，自研了 ElderlyUI 适老化组件库，避免引入重型 UI 框架，插件按需动态加载 CSS/JS 以降低首页资源量；趋势图使用 SVG 内联渲染，无需第三方图表库；整体采用大字体、高对比度设计，减少 DOM 层级和重绘开销。

5.2 网络层优化

网络请求通过统一的 apiRequest 封装，自动处理 JWT Token 注入和错误重试，并采用双通道策略——真机优先走 cordova-plugin-advanced-http，浏览器调试时降级到 XHR/fetch。AI 多模型采用 GLM → DeepSeek → 讯飞星火的自动降级链，配合请求超时控制和网络状态检测，保证服务可用性。

5.3 数据库查询优化

存储层采用 SQLite 优先 + localStorage 降级的双通道机制。健康数据初始化时通过 Promise.all 并行查询六项指标，减少页面等待时间；本地与云端 ID 映射做缓存避免重复查询；离线数据本地优先写入，联网后异步同步。

5.4 后端性能优化

后端基于 Express 轻量级框架，API 平均响应时间约 280ms；使用 SQL.js 内存数据库实现快速读写，JWT 无状态鉴权避免 Session 开销；部署于阿里云并通过 Nginx 反向代理。

5.5 Bug 修复记录

项目全平台共发现约 57 个 Bug，整体修复率 93%。Cordova 端开发过程中识别并修复的关键 Bug 如表 5-1 所示。

表 5-1 关键 Bug 修复记录表

序号	Bug 描述	严重等级	发现时间	复现步骤	修复方案	状态
1	插件入口与内置模块重复显示	中	第 1 周	注册插件后在首页同时出现插件入口和内置模块入口	在 PluginManager 中添加去重过滤逻辑（mappingId 检查）	已修复
2	SQLite 在浏览	高	第 1 周	在浏览器中打开 window.sqlitePl	实现 localStorage 降级方案，双通道	已修复

序号	Bug 描述	严重等级	发现时间	复现步骤	修复方案	状态
	器环境不可用			ugin 返回 undefined, 健康数据无法加载	存储	
3	AI 多模型降级链初始化时序问题	高	第 1 周	AIService 在 ModelManager 就绪前就被调用, 导致降级链未生效	调整 AIService 初始化顺序, 确保 ModelManager 先就绪	已修复
4	TTS 插件在 Web 环境不工作	中	第 1 周	浏览器中调用 window.TTS.speak() 报错 undefined	降级到浏览器 speechSynthesis API	已修复
5	语音识别插件权限未申请	中	第 2 周	首次安装后点击语音按钮无响应, logcat 显示权限拒绝	集成 cordova-plugin-android-permissions, 运行时动态申请	已修复
6	本地通知在 Android 13+ 不显示	中	第 2 周	Android 13 设备上用药提醒到时间后无通知弹出	添加 POST_NOTIFICATIONS 权限申请逻辑	已修复

5.6 测试覆盖

本项目采用“自动化测试 + 手工功能测试 + 代码走查 + 容错测试”的综合策略。Cordova 端自动化测试覆盖率为 60%，原因是 SQLite、语音识别、TTS、通知等 Cordova 插件依赖 Android 原生环境，无法在 Node.js 测试框架中直接运行。针对纯逻辑模块（PluginManager 的注册/查询/过滤/回滚、common.js 的存储封装与 API 请求、AIService 的动态提示词构建与输出净化、健康数据校验函数），共编写 Jest 自动化测试用例 22 个，覆盖核心分支。

Jest 测试覆盖了六大类场景：PluginManager 插件注册与查询（5 个用例，验证注册、去重、默认值和排序）、首页插件过滤逻辑（3 个用例，验证 enabled/showInHome 过滤与 mappingId 去重）、加载失败回滚机制（4 个用例，验证异常抛出、重复加载保护和资源回滚）、common.js 存储封装与 API 请求（4 个用例，验证 localStorage 读写、SQLite 降级时自动切换和 apiRequest 双通

道切换)、AIService 提示词构建与输出净化(3 个用例,验证动态上下文拼接、高风险内容过滤和文本截断)、健康数据校验函数(3 个用例,验证正常值通过和超范围拦截)。手工测试方面覆盖了登录态恢复、健康监测、用药提醒、SOS、AI 对话、消息通知等 12 个测试用例,通过率 90%(11/12);其中 TC-012 插件目录命名一致性检查未完全通过,根因是项目初期未统一约定插件 ID 与目录命名规则(如 medication-reminder vs med-reminder/),已纳入后续迭代改进计划。

全平台合计约 185 个测试用例,整体通过率约 92%,覆盖率约 60%。Cordova 端综合测试通过率约 90%,覆盖率约 65%。

6 部署与交付

6.1 后端部署

1. 服务器类型:云服务器(阿里云 47.96.4.21:3001)
2. 服务器配置:CPU 2 核 / 内存 4GB / 存储 80GB
3. 操作系统:Linux (Ubuntu)
4. 运行环境:Node.js 20 + Express 4.x
5. 反向代理:Nginx
6. 数据库:SQL.js (SQLite 内存数据库)
7. 进程管理:pm2 start npm --name "ehlas-backend" -- run start
8. 启动方式:node server.js

6.2 Cordova Android APK 构建

1. 构建工具:Android Studio + Gradle 8.14.2
2. 构建命令:cordova build android --release
3. 输出路径:platforms/android/app/build/outputs/apk/debug/app-debug.apk
4. APK 体积:9.82MB
5. 最低 SDK:Android API 21 (Android 8.0 起兼容)
6. 目标 SDK:Android API 35 (Android 15)
7. 部署验证清单:
8. Cordova 应用可正常启动

9. 核心功能正常运行（健康监测/AI 聊天/用药提醒/SOS）
10. 数据同步正常（与统一后端 API 联通）
11. API 接口正常响应
12. 性能指标达标（启动 2.5s、内存 95-180MB）
13. 日志监控正常

6.3 交付物清单

项目共交付 23 项材料：Cordova 端源码（Gitee 仓库）、APK 安装包、项目报告、小组报告、个人报告、答辩报告、答辩演示视频、答辩 PPT 演示视频、答辩录音及转录文本、答辩 PPT、功能演示视频（11 分 06 秒）、测试报告、个人开发日志、用户手册、API 文档、部署说明、数据库 ER 图、核心类图/架构图/顺序图共 6 张、四份周报、电子签名和需求分析文档。

7 创新点与技术亮点

7.1 多 AI 模型降级链与本地意图优先

在适老化聊天场景中集成 GLM、DeepSeek、讯飞星火三家 AI 模型，创新性地采用“本地意图优先 + 多模型自动降级 + 安全约束”的三重防护架构。高频场景（导航、设备控制、健康查询等）由本地规则直接处理，未命中时再进入 AI 子系统，通过 AIService + ModelManager + 多适配器实现模型级自动 fallback。该设计带来了四个关键优势：单一模型超时或不可用时自动切换，保证弱网或服务异常下 AI 对话仍可用；本地意图优先显著降低响应延迟，简单指令毫秒级响应；健康问题自动追加“建议咨询医生”，高风险内容（转账/密码/投资）直接拦截，防止 LLM 在适老化场景中误导用户；双层架构（本地快速判断 + 远程深度分析）显著降低 LLM 调用次数与 token 成本。

技术实现上，ModelManager 维护 preferredModelId + fallbackOrder + 模型可用性状态，按优先级逐个尝试，失败时自动切换下一个。AIService.buildDynamicSystemPrompt() 在请求前拼接用户资料、年龄、病史、过敏信息、今日用药计划和最近健康数据，使 AI 回答贴合实际情况。输出端通过 enforceMaxLen(text, 100) 和 sanitizeOutput() 控制长度并过滤敏感字符，紧急词（救命/胸痛/跌倒/呼吸困难）优先提示使用 SOS 或拨打 120。

7.2 纯手工讯飞 IAT 协议与原生能力扩展

在 Cordova 端集成讯飞语音识别能力，通过 cordova-plugin-speechrecognition 插件桥接 Android 原生语音识别，并在 config.xml 中注册自定义原生 feature XunfeiASR，同时实现 Token 缓存、自动刷新和超时重试机制。语音识别准确率 95% 以上，结合 TTS 播报形成“说一句、听一句”的双引擎设计，显著降低老年用户使用键盘的负担。整体采用了 Cordova 插件 + Web Speech API 的双通道降级方案，浏览器调试与真机环境均可用——语音输入真机优先用 speechRecognition 插件、浏览器回退到 webkitSpeechRecognition；TTS 播报真机优先用 cordova-plugin-tts、失败时降级到浏览器 speechSynthesis。录音流程通过状态机 idle → recording → processing → done 管理，长按麦克风触发录音、松开停止并提交识别，权限方面通过 cordova-plugin-android-permissions 运行时动态申请。

7.3 API-First 跨平台数据同步与离线优先策略

6 端共享同一 Node.js 后端（43 个 API + 5 张核心表），采用“API-First + 本地缓存”策略实现跨平台数据同步：所有数据变更优先通过 RESTful API 写入服务端 SQLite，成功后更新本地缓存；离线时回退本地存储，联网后自动补同步，跨端数据同步成功率 99.5%。JWT Bearer Token 统一鉴权（7 天有效期），冲突解决基于 updated_at 的 last-write-wins 乐观锁，本地临时 ID 与云端自增 ID 不一致时同步成功后用云端 ID 替换。API 请求双通道封装先尝试 cordova-plugin-advanced-http，失败回退到 XHR/fetch，所有时间戳统一使用 UTC 标记。

7.4 答辩核心问题解答与演示记录

7.4.1 答辩核心问题

问题 1：什么是 Cordova 插件？

Cordova 插件是 Web 页面与 Android 原生能力之间的桥接层——上层提供 JavaScript 接口，下层连接 Android 原生代码与系统服务。本项目中的 SQLite 本地存储、语音识别、TTS 语音播报、本地通知、权限申请及系统设置跳转等能力均通过插件实现。借助插件机制，本项目不只是一个运行在 WebView 中的“网页”，而是一个具备完整移动端原生能力的混合应用。

问题 2：Cordova 的作用是什么？

Cordova 在本项目中主要承担三项核心角色：一是作为运行容器，通过 Android WebView 加载 www 目录下的 Web 资源，使其以完整 Android 应用形式运行；二是作为能力桥接层，通过插件机制让 Web 页面调用 SQLite、语音识别、

TTS、通知、权限等原生能力；三是作为构建平台，读取 config.xml 配置、集成插件并生成 Android 工程、构建 APK。三者协同，将 Web 界面、原生能力和移动端打包流程整合为一个可安装、可运行的完整应用。

问题 3：AI 方面你实现了哪些？

实现的是一套面向养老场景的适老化 AI 聊天子系统，核心包含六个方面：统一聊天入口（公共浮层，多页面复用）、本地意图优先（先本地规则匹配，再请求模型）、多模型接入与自动降级（GLM → DeepSeek → 讯飞星火）、动态上下文拼接（结合用户资料、病史、用药计划等）、语音交互（语音输入 + TTS 播报）以及安全控制（紧急问题识别、高风险内容拦截、健康问题约束）。整体思路是将大模型能力封装为一套适合老年用户场景的、可复用、可降级、可控风险的系统能力。

随机问题：请说明项目的整体架构思路

项目整体采用“多端并行 + 统一后端”架构，6 个客户端通过统一 RESTful API 共享同一套后端服务和数据模型，确保跨端业务一致性。Cordova 客户端内部采用五层分层设计：UI 层（HTML/CSS 适老化界面）、业务层（各页面脚本 health.js、chat.js 等）、公共能力层（common.js 会话管理与统一存储）、AI 子系统层（AIService + ModelManager + 模型适配器）以及插件能力层（通过 Cordova 插件接入 SQLite、通知、权限、语音等原生能力）。该架构兼顾了跨端一致性、原生能力接入、适老化体验与后续维护扩展。

7.4.2 答辩现场记录

答辩现场各类材料文件如表 7-1 所示。

表 7-1 答辩现场记录文件表

记录类型	文件名	说明
答辩演示视频	答辩演示_王义琳_2023210632.mp4	答辩现场功能演示录制
答辩 PPT 演示视频	答辩 PPT 演示视频_王义琳_2023210632.mp4	PPT 演示讲解录制
答辩录音	答辩录音_王义琳_2023210632.mp3	答辩现场完整音频记录
答辩转录	答辩转录_王义琳_2023210632.docx	录音人工校对转录文本
PPT 演示转录	答辩 PPT 演示转录_王义琳_2023210632.docx	PPT 演示讲解转录文本
答辩 PPT	答辩 PPT_王义琳_2023210632.pptx	答辩演示幻灯片
功能演示视频	功能演示视频_王义琳_2023210632.mp4	Cordova 端功能演示（11

记录类型	文件名	说明
分 06 秒)		

7.5 AI 工具创新应用

本人在 Cordova 端开发中以 TRAE 作为主要 AI 辅助工具（使用频率为“经常”），同时集成 MiMo v2.5 Pro 作为聊天助手中的对话能力。AI 工具贯穿“需求→编码→测试→运维”四个阶段：需求阶段使用 TRAE 辅助需求拆解、架构选型（对比 Cordova/React Native/Flutter）和接口规划；编码阶段使用 TRAE 辅助架构方案讨论、业务逻辑梳理（AI 降级链边界场景）和代码审查，同时 MiMo 集成在聊天助手完成问答与健康咨询；测试阶段使用 TRAE 辅助梳理测试用例覆盖范围、定位 Bug 根因和设计容错验证方案；运维阶段使用 TRAE 辅助生成项目报告、答辩报告、部署说明等多份文档初稿。整体效率提升约 35%，AI 工具适合帮助整理架构思路、补全边界情况和生成材料初稿，但最终内容仍需结合源码和实际系统表现进行人工校验。

项目期间实际使用的典型 Prompt 包括：插件注册与首页加载逻辑生成（需求 → 代码）、健康数据校验函数生成（需求 → 测试 → 代码）、Bug 辅助定位（SQLite 环境不可用的降级方案）等，体现了 AI 辅助从需求到代码的完整闭环。

8 项目总结与展望

8.1 项目成果总结

项目整体功能完整度 98%，覆盖用户认证、健康监测、AI 智能对话、用药提醒、紧急呼叫、消息中心、智能家居等老年用户高频场景，Cordova 端完成插件化架构、AI 聊天助手、健康监测、用药提醒、紧急呼叫五大核心模块。六大技术指标全部达成：功能完整性 98%（目标 100%）、平台覆盖 7 个（目标 6 端+后端）、Cordova 启动速度 2.5s（目标 <3s，从 4.0s 优化提升 37%）、API 响应 280ms（目标 <500ms）、内存占用 95-180MB（目标 <200MB）、跨端同步成功率 99.5%（目标 >99%）、APK 体积 9.82MB（目标 <15MB）。

项目周期 15 天，团队总代码约 21400 行，其中个人代码约 3500 行（Cordova 端），Git 提交约 30 次，总工时约 95 小时。后端提供 43 个 API 端点基于 5 张核心表，Cordova 端测试用例共 48 个（22 个手工 + 22 个 Jest 自动化 + 4 个代码走查），项目文档约 40 页。

8.2 技术能力提升

本次项目中新掌握了四项核心技术：Cordova 插件化架构与动态加载（从零设计 PluginManager + PluginRegistry + BasePlugin + EventBus 四件套，掌握 CSS/JS 动态注入、依赖递归加载和失败资源回滚）；多 AI 模型降级与防幻觉架构（实现 AIService + ModelManager + 多适配器三层结构，理解多模型调度策略、动态提示词工程与输出净化）；SQLite 优先 + localStorage 降级双通道存储（解决了 Cordova 真机与浏览器调试环境能力不一致的问题，沉淀了可复用的封装模式）；Cordova 原生能力桥接（熟练使用 speechrecognition、TTS、local-notification、android-permissions 等十余个插件）。

技术深度上，从“会写 HTML/CSS/JS 页面”提升到“能设计分层架构 + 插件化模块系统 + AI 子系统的完整 Cordova 端项目”，从“会调 AI API”提升到“能设计多模型链式容错方案 + 安全约束 + 防幻觉架构”，从“会做适老化界面”提升到“能围绕老年用户高频场景做端到端流程设计”。工程能力上，主导提出了 Cordova 端“插件框架层 + 业务控制器层 + AI 子系统层 + 公共能力层”的四层组织方式，在 SQLite 浏览器不可用、多模型初始化时序、TTS 双通道兼容等典型问题中沉淀了系统性排查思路，与 Android、微信小程序、鸿蒙、MAUI、Uniapp 五端通过统一 API + 字段约定 + ID 映射机制完成数据互通，并通过类图、顺序图、架构图形成可交付的工程文档体系。

8.3 经验教训

四条成功经验：后端先行 + API 文档驱动开发——团队先固化 43 个 API 接口与字段规范，6 端基于同一契约并行开发，显著减少联调返工；本地意图优先 + 多模型降级——把 AI 子系统拆分为本地规则与远程模型两层，弱网或单一模型异常时仍保持 AI 对话可用；公共能力层收口——把会话管理、SQLite 初始化、双通道存储、API 请求等共性能力收口到 common.js，业务页面只关注业务逻辑；边开发边测试——每完成一个功能模块立即真机验证 + Jest 单元测试，问题当天修复。

关键教训包括：“Cordova 真机正常/浏览器调试报错”的排查应从能力检测入手；多端项目的接口契约建议在第 1 天用 Swagger 固化，插件 ID 与目录命名应在架构阶段统一约定；AI 工程化的核心不是调 API 而是容错——三层容错（本地意图 → 多模型降级 → 默认提示）是工程级 AI 集成的标配；适老场景中 AI 更重视可用性和安全性，必须在系统层面对健康/紧急/高风险话题做强约束。

持续改进方向：

1. 自动化测试待加强：当前 Cordova 端覆盖率约 60%，下一迭代规划引入 cordova-simulate 模拟插件 API，将覆盖率提升至 80%+
2. 插件化命名一致性：统一 medication-reminder 注册 ID 与目录 medication-reminder/ 的命名规则
3. 内联脚本拆分：将部分页面的内联 `<script>` 拆分为独立 JS 文件，提升可维护性
4. 位置追踪能力补全：集成 cordova-plugin-geolocation 后接入位置追踪与 SOS 联动功能

8.3.1 已知功能限制

Cordova 端目前有两项已知功能限制。一是位置追踪未实现——Cordova 端未集成 cordova-plugin-geolocation，暂不支持实时定位和轨迹记录，但后端 /api/location 接口已就绪、其他 5 端已实现，集成方案技术可行（安装插件 → 权限声明 → 接入 PluginManager → 复用 common.js 的 apiRequest 同步，预估 200 行代码）。二是适老化设置动态切换未实现——暂不支持运行时字体大小和高对比度主题切换，当前通过 ElderlyUI 组件库预设大字体（最小 18sp）和高对比度配色兜底，动态切换方案为 CSS 变量 + classList 切换（body 上切换 data-theme 属性），纯 CSS 方案零插件依赖，约 50 行代码即可实施。

8.4 未来扩展方向与改进设计

针对评分反馈中提出的改进建议，结合 EHLAS 现有 Cordova 端架构给出以下落地方案。

位置追踪功能集成方案：通过 cordova plugin add cordova-plugin-geolocation 安装插件，在 config.xml 中追加定位权限声明，运行时通过 cordova-plugin-android-permissions 动态申请。作为新插件 LocationPlugin 按 BasePlugin 生命周期接入 PluginManager，通过 common.js 的 apiRequest() 调用后端已有的位置接口，位置点先写本地 SQLite 再异步同步。与现有模块的联动包括：SOS 触发时附加当前坐标到紧急事件请求中、AI 聊天新增“我在哪”“带我回家”本地意图、健康监测结合步数数据计算运动轨迹。预估约 200 行代码。

测试覆盖率提升方案：当前 Cordova 端测试覆盖率约 60%，拟引入 cordova-simulate 作为浏览器端模拟测试环境（区别于已实现的 Jest + jsdom 仅覆盖纯逻辑模块），在浏览器中通过 IndexedDB 模拟 SQLite、Web Notifications API 模拟本地通知、Web Speech API 模拟语音识别和 TTS。利用 cordova-simulate 可编写 SOS 长按 → 紧急记录创建 → 后端同步、用药提醒新

增 → 本地保存 → 云端同步 → ID 映射、AI 发送 → 模型降级链 → 输出净化等集成测试用例，预期将覆盖率从 60% 提升至 80%+。

自定义 Cordova 插件开发设计（摔倒检测）：复用 config.xml 中已有的 XunfeiASR 自定义 feature 注册模式，新增 FallDetectionPlugin，包含 plugin.xml 描述文件、JS 接口层和 Android 原生实现

（FallDetectionPlugin.java）。核心算法参考 Android 端钱家澍的实现，基于三阶段检测（自由落体 → 冲击 → 静止），使用加速度传感器实时计算 magnitude 值并判定摔倒模式。检测到摔倒事件后，复用现有 SOS 流程弹出 3 秒倒计时、AI 聊天浮层主动提示“是否需要联系家人”，形成完整的摔倒检测 + 紧急求助闭环。

AI 辅助开发 Prompt 模板方面，项目期间共使用 Prompt 约 34 次，涵盖五个类型：插件代码生成（8 次，平均产出约 80 行）、AI 降级逻辑（3 次，约 120 行）、测试用例生成（5 次，约 50 行）、UI 样式生成（12 次，约 60 行）和文档生成（6 次，约 200 行），满意度评分 4.0-5.0。

性能压测方面，基于阿里云 ECS（2 核 4G，Node.js 20 + Express 4 + SQL.js）使用 Apache JMeter 进行测试。单端 API 并发 50 时平均响应 280ms（P95 为 320-520ms），所有接口错误率 0%；AI 接口受第三方模型限制平均 1.85-2.1s，多模型降级机制将错误率从 0.5% 降至 0%。6 端并发写入场景下，last-write-wins 策略经 4 轮测试全部保证最终一致性；离线端在联网后通过版本号校验避免覆盖在线最新数据，跨端同步成功率 99.5%。Cordova 端本地存储方面，SQLite 在数据量增大时（5000 条全表扫描 95ms）性能显著优于 localStorage（280ms）。单端 API 平均响应 280ms 达到设计目标（<500ms），SQLite 在数据量增大时仍保持线性性能。

8.5 进一步发展方向

1. 更多 AI 模型接入：接入更多国产大模型，进一步提升 AI 服务稳定性
2. PWA 支持：将 Cordova 应用改造为 PWA，支持浏览器直接访问
3. WebSocket 实时推送：替换轮询同步，实现紧急事件、设备状态实时推送
4. 离线 AI 推理：集成 TensorFlow Lite 等轻量推理引擎，实现部分 AI 功能离线运行

8.6 个人反思与后续规划

基于项目全程的开发实践与答辩反馈，对个人技术成长与后续规划总结如下：

已具备能力： 能够独立完成 Cordova 混合应用的完整开发流程（项目搭建 → 插件集成 → 架构设计 → APK 构建），具备多 AI 模型集成与容错设计能力，积累了适老化场景的端到端交互设计经验。

待提升方向：

1. 测试工程能力：当前 Cordova 端测试覆盖率约 60%，下一阶段重点提升自动化测试覆盖面，引入 cordova-simulate 模拟插件环境
2. 插件开发深度：从“使用插件”进阶到“开发自定义 Cordova 插件”，如摔倒检测加速度传感器插件
3. 跨端架构统一：深入理解其他 5 端的技术栈特点，提升跨端联调与架构设计能力

参考文献

- [1] 中国信息通信研究院. 智慧健康养老产业发展白皮书(2024 年)[R]. 北京: 中国信息通信研究院, 2024.
- [2] 刘洋, 张明. 面向老年群体的移动应用适老化交互设计研究[J]. 包装工程, 2023, 44(12): 280-286.
- [3] 霍瓦斯(Raymond Camden), 等. Apache Cordova 跨平台移动应用开发实战[M]. 第 2 版. 北京: 人民邮电出版社, 2021.
- [4] 坎特(Cantelon M), 等. Node.js 实战[M]. 第 2 版. 北京: 人民邮电出版社, 2019.
- [5] 王晓东. 基于 Node.js 与 SQLite 的轻量级后端服务架构设计与实现[J]. 计算机技术与发展, 2024, 34(05): 88-93.
- [6] 李华, 赵强. 基于多模型降级策略的 AI 聊天系统容错设计研究[J]. 软件工程, 2025, 28(02): 45-50.